Programming Fundamentals

- Lecturer: Dr. Arnold Meijster
- e-mail: a.meijster@rug.nl
- room: 5161.343 (Bernoulliborg)



Functions

A *function* is a program fragment that

- can be used from any other place in the program (‘function call’).
- performs a specific task.
- is logically coherent.
- can return a result (value).

Function definition

The definition of a function consists of its prototype + the *body* that implements the function.

General form:

```
<returntype> functionName(<type1> parameter1, …,
                           <typeN> parameterN) {
   /* body of the function */
}
```

`<returntype>` is the type of the function result.
The return type is `void` if the function does not return a result.

We use the following conventions for choosing function names:

- camelCase
- No result (void): imperative form, e.g. `void printValue(int n)`
- With result: description of result, e.g. `int isPrimeNumber(int n)`.

The parameters of a function can be used in the body of the function as if they were declared and initialised as *local variables*.

Function call

Once a function is defined, it can be used anywhere in the program (‘function call’).

A function call consists of the name of the function and a list of *arguments* (between parentheses).

Each argument must be an *expression* of the right type. The *values* of the expressions are used to ‘initialise’ the parameters of the function.

The call of a void-function is a ‘normal’ statement, e.g. `printValue(n)`.

The call of a function that returns a result is an expression having the same type as the return-type of the function, e.g. `x = y + 3*someFunction(n)`.

Example (another version)

```
int maximum(int a, int b) {
  return (a > b ? a : b);
}

void printMaximum(int p, int q, int r, int s) {
  int x = maximum(p, q);
  int y = maximum(r, s);
  printf("%d\n", maximum(x,y));
}

int main(int argc, char *argv[]) {
  printMaximum(1, 2, 3, 4);
  return 0;
}
```

Scope

- The *scope* of a variable is the region in the code in which the variable ‘exists’.

- The scope of a variable is the region that starts at the declaration of the variable and ends at the closing bracket } of the corresponding code block.
  - Exception 1: parameters of a function are declared outside a code block: their scope is the entire function.
  - Exception 2: ‘*global variables*’ (see next item)

- *Global variables* are variables that are declared outside a function. They are (usually) declared at the top of a program. Their scope is the entire program.
  - Advice: use global variables only if really needed.

- Variables that are declared in a function or within a code block (i.e. between a pair of brackets) are called ‘*local variables*’. Their scope is the code block that they are declared in.

- This may lead to a phenomenon called ‘*shadowing*’. A local variable makes a variable (from a surrounding scope) with the same name temporarily invisible.

```c
#include <stdio.h>

int x=1; /* global x */

void print() {
   printf("%d\n", x);
}

void printX(int x) {/* local x (shadows global x) */
   printf("%d\n", x);
}

void someFunction() {
   int x = 3; /* local x (shadows global x) */
   print();
   printX(x);
   {
     int x = 4; /* local x (shadows surrounding local x) */
     print();
     printX(x);
   }
   printX(x);
}

int main(int argc, char *argv[]) {
  int x = 2; /* local x (shadows global x) */
  print();
  printX(x);
  printf("%d\n", x);
  someFunction();
  printf("%d\n", x);
  return 0;
}
```

Output:

1

2

2

1

3

1

4

3

2

```c
#include <stdio.h>

int x=1; /* global x */

void print() {
   printf("%d\n", x);
}

void printX(int x) {/* local x (shadows global x) */
   printf("%d\n", x);
}

void someFunction() {
   x = 3;   // was: int x = 3;
   print();
   printX(x);
   {
     int x = 4; /* local x (shadows surrounding local x) */
     print();
     printX(x);
   }
   printX(x);
}

int main(int argc, char *argv[]) {
  int x = 2; /* local x (shadows global x) */
  print();
  printX(x);
  printf("%d\n", x);
  someFunction();
  printf("%d\n", x);
  return 0;
}
```

Try to figure out the output yourself!

Prime factorization

Exercise: write a program that repeatedly asks the user to type in an integer >1 (or 0,1 to stop) and then prints the prime factorization of the input number.

```
Please, type a natural number >1 (<=1 means stop): 123456
2^6 * 3^1 * 643^1
Please, type a natural number >1 (<=1 means stop): 997
997^1
Please, type a natural number >1 (<=1 means stop): 64
2^6
Please, type a natural number >1 (<=1 means stop): 192
2^6 * 3^1
Please, type a natural number >1 (<=1 means stop): 1
Goodbye.
```

Prime factorization

```
int main(int argc, char *argv[]) {
  int number = readNumber();
  while (number > 1) {
    printFactorization(number);
    number = readNumber();
  }
  printf("Goodbye.\n");
  return 0;
}
```

readNumber

```
int readNumber() {
  int number;
  do {
    printf("Type a natural number > 1 (or 0,1 to stop): ");
    scanf("%d", &number);
  } while (number < 0);
  return number;
}
```

printFactorization

```
void printFactorization(int number) {
  int divisor = 2;
  int first = 1;  /* 1 == TRUE */
  while (number != 1) {
    int exponent = countFactors(number, divisor);
    if (exponent > 0) {
      if (first) {
        first = 0;
      } else {
        printf(" * ");
      }
      printf ("%d^%d", divisor, exponent);
      number /= exponentiation(divisor, exponent);
    }
    divisor++;
  }
  printf("\n");
}
```

printFactorization (efficient)

```
void printFactorization(int number) {
  int divisor = 2;
  int first = 1;  /* 1 == TRUE */
  while (number != 1) {
    int exponent = countFactors(number, divisor);
    if (exponent > 0) {
       if (first) {
         first = 0;
       } else {
         printf(" * ");
       }
       printf ("%d^%d", divisor, exponent);
       number /= exponentiation(divisor, exponent);
    }
    divisor++;
  }
} if (number > 1) {
    printf((first ? "%d^1" : " * %d^1"), number);
  }
  printf("\n");
}
```

printFactorization: Efficieny analysis

Let us compute `printFactorization(int 999983);`

The number 999983 is the largest prime below a million.

The initial version of the program would require 999983 iterations.

The second (more efficient) version would require $\sqrt{999983} \approx 1000$ iterations. A major improvement! This algorithm is (for this input) about 1000 times faster.

countFactors

The function call `countFactors(x,d)` returns the number of times that we can divide `x` by `d`.

```
int countFactors(int n, int d) {
  int cnt=0;
  while (isDivisor(n, d)) {
    n /= d;
    cnt++;
  }
  return cnt;
}
```

isDivisor

The function call `isDivisor(x,d)` returns true (i.e. 1) if `x` is an integer multiple of `d`.

```
int isDivisor (int x, int d) {
  return (x%d == 0);
}
```

Exponentiation (see week 2)

```
int exponentiation(int number, int exponent) {
  /* returns number^exponent */
  int m=1;
  while (exponent!=0) {
    if (exponent%2 == 0) {
      number = number*number;
      exponent = exponent/2;
    } else {
      m = m*number;
      exponent--;
    }
  }
  return m;
}
```

Call-by-value

What is the output of this program?

```
void nextInt(int n) {
  n++;
  printf("%d\n", n);
}

int main(int argc, char *argv[]) {
  int n = 1;
  printf("%d\n", n);
  nextInt(n);
  printf("%d\n", n);
  return 0;
}
```

```
Output:

1

2

1
```

Call-by-value

We make a distinction between *formal* and *actual* parameters:

- Formal parameters: *parameter names* from the prototype of a function
- Actual parameters: *values* that are passed to a function

Call-by-value: When we call a function, the *value of expressions* that are passed to the function are *copied* into the formal parameters before execution of the function.

A value is not a variable. Hence, it cannot change!

Conclusion: Due to call-by-value it is not possible to change the value of actual parameters!

Call-by-value

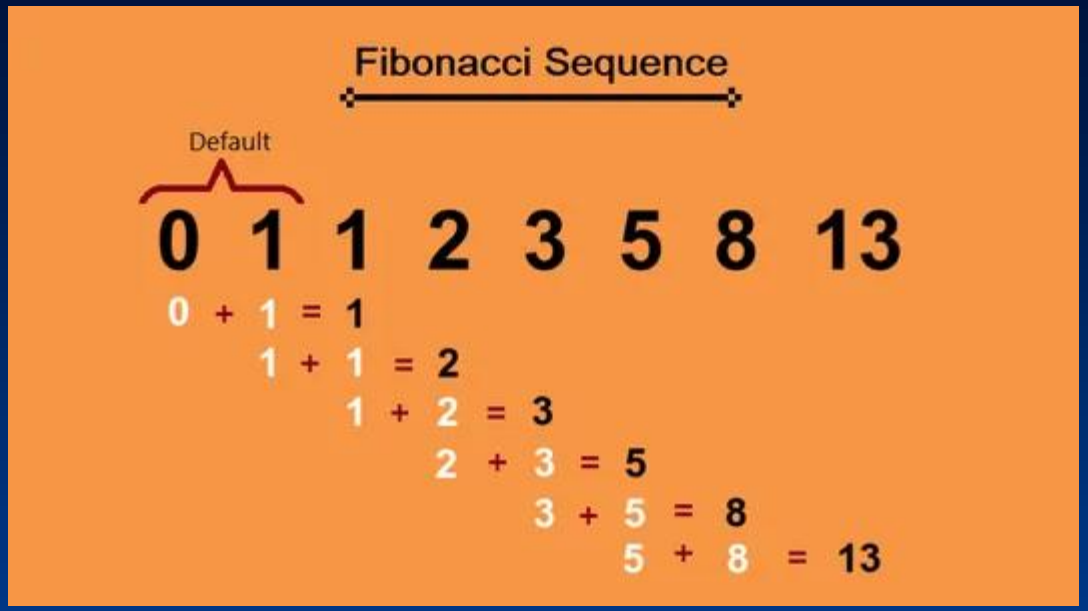
```
void nextInt(int n) {
  n++;
  printf("%d\n", n);
}

int main(int argc, char *argv[]) {
  nextInt(1); // We pass a value! 1++ is meaningless
  return 0;
}
```



```
int main(int argc, char *argv[]) {
  int n = 1;
  nextInt(n); // same here
  return 0;
}
```

Fibonacci again: 0, 1, 1, 2, 3, 5, …




```
int fib(int n) {
  int a = 0, b = 1;
  for (int i=0; i < n; i++) {
    b = a + b;
    a = b - a;
  }
  return a;
}
```

Fibonacci: one, one, two, three, five, …

In which year do we reach a number of pairs that is at least (a given) **n**?

```
int young = 1;
int mature = 0;
int year = 0;
int n;
scanf("%d", &n);
while (young + mature < n) {
  mature = mature + young;
  young = mature - young;
  year++;
}
/* here: young + mature >= n  (logical negation of guard) */
printf("In year %d we have at least %d pairs.\n", year, n);
```



The code below is easier to understand, but less efficient due to repeated recomputation!

```
int year = 0;
while (fib(year) < n) {
  year++;
}
printf("In year %d we have at least %d pairs.\n", year, n);
```

Function calling another function: Palindromic Number

Write a function `isPalindromicNumber(int n)` that returns 1 if `n` is a palindromic number, otherwise it returns 0.

- `isPalindromicNumber(121)` returns 1
- `isPalindromicNumber(123321)` returns 1
- `isPalindromicNumber(1231)` returns 0

```
int isPalindromicNumber(int n) {
  return (reverseInt(n) == n); // returns 0,1(false,true)
}
```

reverseInt function

```
int reverseInt(int n) {
  int reverse = 0;
  while (n != 0) {
    int remainder = n%10;
    reverse = reverse*10 + remainder;
    n /= 10;
  }
  return reverse;
}
```

n	n != 0	remainder	reverse
2345	true	5	0 * 10 + 5 = 5
234	true	4	5 * 10 + 4 = 54
23	true	3	54 * 10 + 3 = 543
2	true	2	543 * 10 + 2 = 5432
0	false	-	Loop terminates.

Old lab exercise: number anagram

A non-negative integer is said to be an *anagram* of another non-negative integer if it can be made equal to the other number by shuffling digits. For example, 12345, 13254, 54321, and 34521 are all anagrams of each other.

The number 123450 is a number anagram of 123045 but it is not an anagram of 012345 because leading zeros are ignored.

Write a program that reads from the input two non-negative integers a and b and outputs `YES` if these numbers are anagrams, or `NO` otherwise. You may assume that a and b fit in a standard `int`.

Example 1:
input:
`12345 54321`
output:
`YES`

Example 2:
input:
`012345 543210`
output:
`NO`

Example 3:
input:
`10293810 29381011`
output:
`NO`

```
int main(int argc, char *argv[]) {
  int a, b;
  scanf("%d %d", &a, &b);
  int digit = 0;
  while ((digit < 10) && (digitCount(digit, a) == digitCount(digit, b))) {
    digit++;
  }
  printf(digit == 10 ? "YES\n" : "NO\n");
  return 0;
}
```

digitCount function

```
int digitCount(int digit, int n) {
   int cnt=0;
   while (n != 0) {
      cnt += (n%10 == digit);
      n /= 10;
   }
   return cnt;
}
```

Old lab exercise: pompous number

A *pompous number* is so full of itself, that it describes itself. The first digit of a pompous number specifies how many 0's there are in the decimal notation of the number, the next digit specifies how many 1's there are, etc. For example, the number 1210 is pompous because there is 1 zero, 2 ones, 1 two, and 0 threes.

Write a program that accepts on its input a positive `int` and outputs `POMPOUS` if this number is pompous, or `MODERATE` otherwise.

Example 1:
input:
`1210`
output:
`POMPOUS`

Example 2:
input:
`1201`
output:
`MODERATE`

Example 3:
input:
`2020`
output:
`POMPOUS`

```
int main(int argc, char *argv[]) {
  int n;
  scanf("%d", &n);
  printf(isPompousNumber(n) ? "POMPOUS\n" : "MODERATE\n");
  return 0;
}
```

Old lab exercise: pompous number

```
int isPompousNumber(int n) {
  int mask = maxPower10(n);
  int m=n, digit = 0;
  while (mask != 0) {
    if (digitCount(digit, n) != m/mask) {
      break;
    }
    m %= mask;
    mask /= 10;
    digit++;
  }
  return (mask == 0);
}
```

Old lab exercise: pompous number

```
int maxPower10(int n) {
   int mp = 1000000000;
   // maximum int has 10 digits
   while (n/mp == 0) {
      mp /= 10;
   }
   return mp;
}
```

Arrays

An *array* is used to store a *sequence* of values of the same type.

An array is a numbered list.

- A member of the array is called an *element*.
- Each element is a ‘variable’ of the corresponding type.

The number/location of an element is called its *index*.

Arrays: Declaration/initialisation

There are three ways to declare an array:

- `type arrayName[N];`
- `type arrayName[N] = {value1, value2, …, valueN};`
- `type arrayName[] = {value1, value2, …, valueN};`

In the first case, the array is declared, but not initialised.

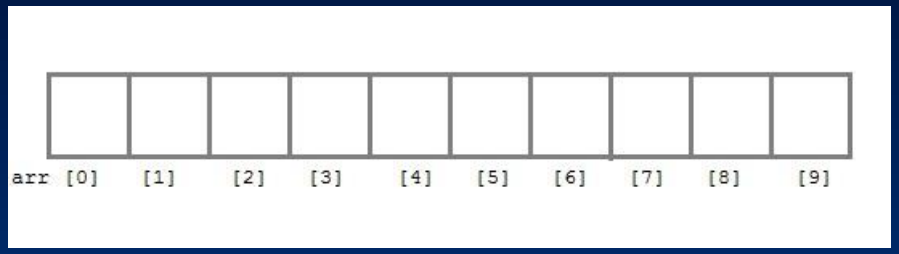
In the second case, the array is declared and initialised.

In the third case, the arrays is declared and initialised as well. Since we give an explicit list of values, the compiler is able to infer the correct size of the array.

Examples

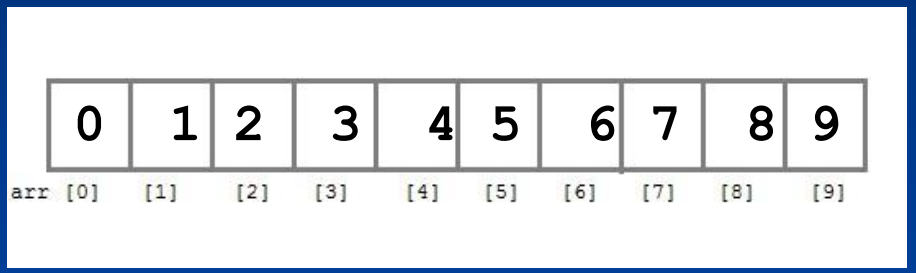
First form:

```
int arr[10];
```



Second form:

```
int arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```



Third form:

```
int arr[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

Array-indexing

Accessing an array-element is called *indexing*.

The element with index `i` from an array `a` is `a[i]`.

The first element has index 0.

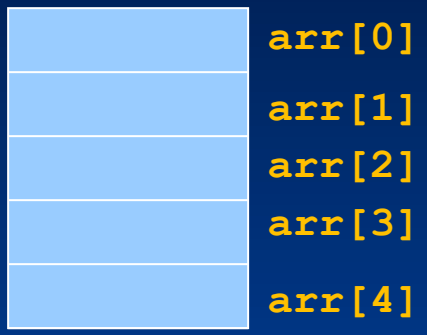
So, the last element of an array of length `n` is `a[n-1]`.

Assigning values to an array

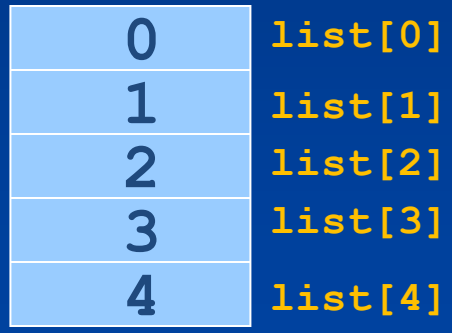
For loops are often used to assign values to an array

Example:

```
int arr[5];
```




```
for (int i=0; i < 5; i++) {
   arr[i] = i;
}
```



Out of bounds: segmentation fault

An array with size `n` has no element with index `n`!

Accessing `a[i]`, where `i<0` or `i>=n`, is an error: **`segmentation fault`**.

```
int a[10];
for (int i=0; i <= 10; i++) {
  a[i] = i;
}
```

C does not check bounds on arrays!

Accessing `a[10]` may give a segmentation fault or overwrite other memory locations.

Example: smallest element

```
#define N 100
int arr[N];
```

Let us assume that the array is already filled with data.

Assignment: determine the smallest element of the array and the index(es) at which it is found.

Example: smallest element

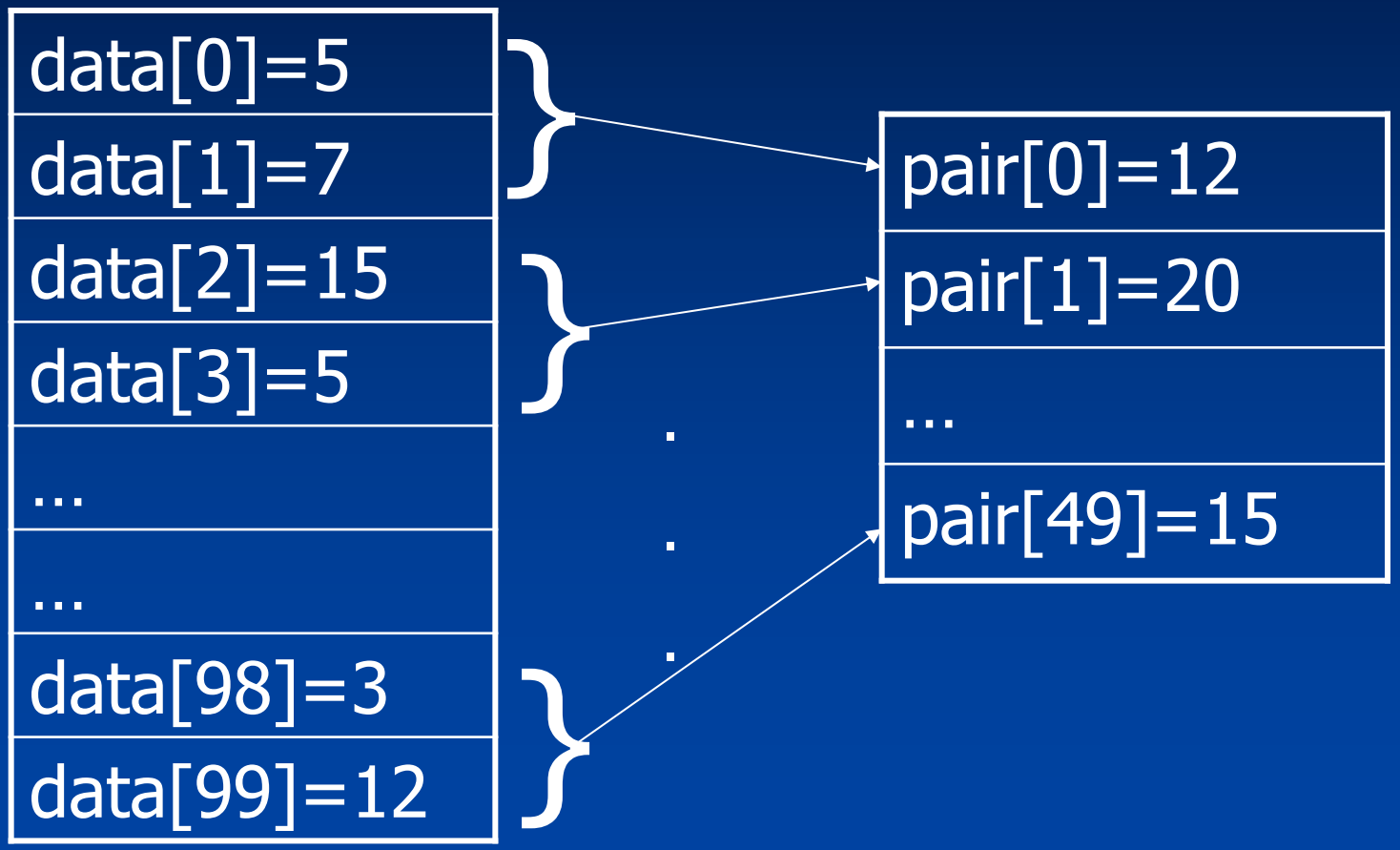
```
int minimum = arr[0];
for (int index=1; index < N; index++) {
  if (arr[index] < minimum) {
    minimum = arr[index];
  }
}

printf("Minimum: %d\n", minimum);

printf("Location(s):");
for (int index=0; index < N; index++) {
  if (arr[index] == minimum){
    printf(" %d", index);
  }
}
printf("\n");
```

Example: pair sum

Find sum of every pair in **`data`** and write in into the **`pair`** array.



Example: pair sum

```
int data[100], pair[50];
/* we assume that data contains values */

for (int i=0; i<50; i++){
   pair[i]= data[2*i] + data[2*i+1];
}
```

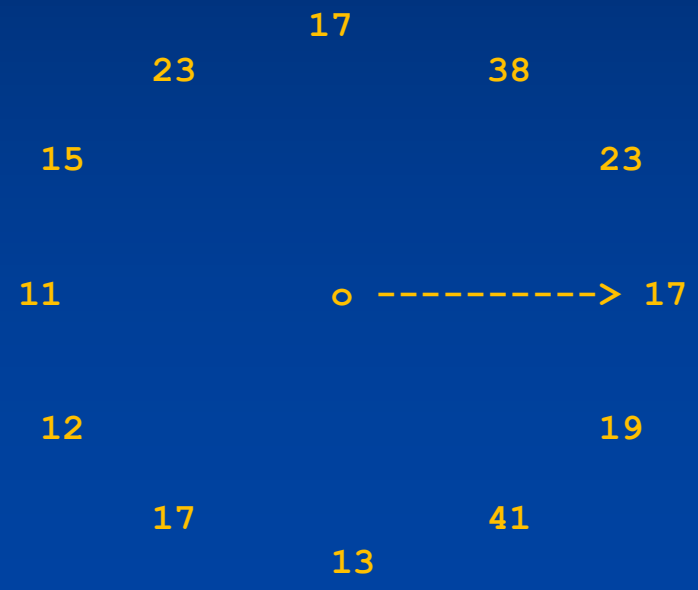
An alternative is:

```
for (int i=0,j=0; i<50; i++,j+=2){
   pair[i]= data[j] + data[j+1];
}
```

Example: wheel of fortune

Given a circle consisting of 12 squares, each containing an integer. Furthermore, there is an arrow that points to exactly one of the squares.

Assignment: write a program fragment that moves the arrow clockwise such that it points to the next square that contains the same value as the square that it initially pointed to.

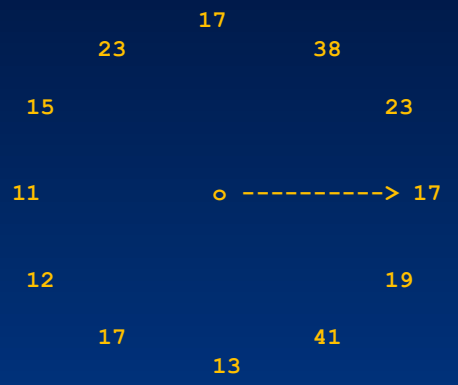


Example: wheel of fortune

We represent the wheel using an array:

```
int wheel[12]={17, 38, 23, 17, 19, 41, 13, 17, 12, 11, 15, 23};
```

The arrow is represented by an integer: `int arrow`.




```
initialValue = wheel[arrow];
arrow = (arrow + 1)%12;
while (wheel[arrow] != initialValue) {
  arrow = (arrow + 1)%12;
}
```

Note the replication of the statement `arrow = (arrow + 1)%12;`

Hence, a do-while-statement may be a more elegant choice:

```
initialValue = wheel[arrow];
do {
  arrow = (arrow + 1)%12;
} while (wheel[arrow] != initialValue);
```

Call by value: swapping ints

The function **swapInts** DOES NOT swap two int variables. The function is valid C, but it is a useless function: it swaps the formal parameters a and b, not the actual parameters!

```
void swapInts(int a, int b) {
  printf("a=%d, b=%d\n", a, b);
  int h = a;
  a = b;
  b = h;
  printf("a=%d, b=%d\n", a, b);
}

int main(int argc, char *argv[]) {
  int a = 1, b = 2;
  swapInts(a,b);
  printf("a=%d, b=%d\n", a, b);
  return 0;
}
```

```
Output:

a=1, b=2

a=2, b=1

a=1, b=2
```

Passing an array to a function

But, the function **swapArray** DOES swap the two int values that are stored in the array **a[]**.

```
void swapArray(int a[2]) {
  printf("a[0]=%d, a[1]=%d\n", a[0], a[1]);
  int h = a[0];
  a[0] = a[1];
  a[1] = h;
  printf("a[0]=%d, a[1]=%d\n", a[0], a[1]);
}

int main(int argc, char *argv[]) {
  int a[2] = {1,2};
  swapArray(a);
  printf("a[0]=%d, a[1]=%d\n", a[0], a[1]);
  return 0;
}
```

```
Output:
a[0]=1, a[1]=2
a[0]=2, a[1]=1
a[0]=2, a[1]=1
```

Intermezzo: Pointers

A **pointer** is a variable that contains a memory address (location in the memory of the computer).

A pointer has a name, a value and a type.

It is important to realise that there is a difference between the value of a pointer (an address) and the value to which it points (in this example 24, an `int`).



Declaring and Casting Pointers

For each data type T you can define a pointer of the type "pointer to T":

`int *p;`	pointer to an `int` (i.e. an `int`-pointer)
`char *q;`	pointer to a `char` (i.e. a `char`-pointer)
`float **w;`	pointer to a `float`-pointer

Dereferencing Pointers (indirection operator)

`int *p;` `p` is an `int`-pointer

`*p` is the content of the memory location that `p` points to. So, `*p` is a 'normal' `int` variable.




```
*p = 24;
```

Address Operator: &

`int i = 24;` `i` is a normal `int` variable

`&i` is an `int`-pointer: the address of the variable `i`

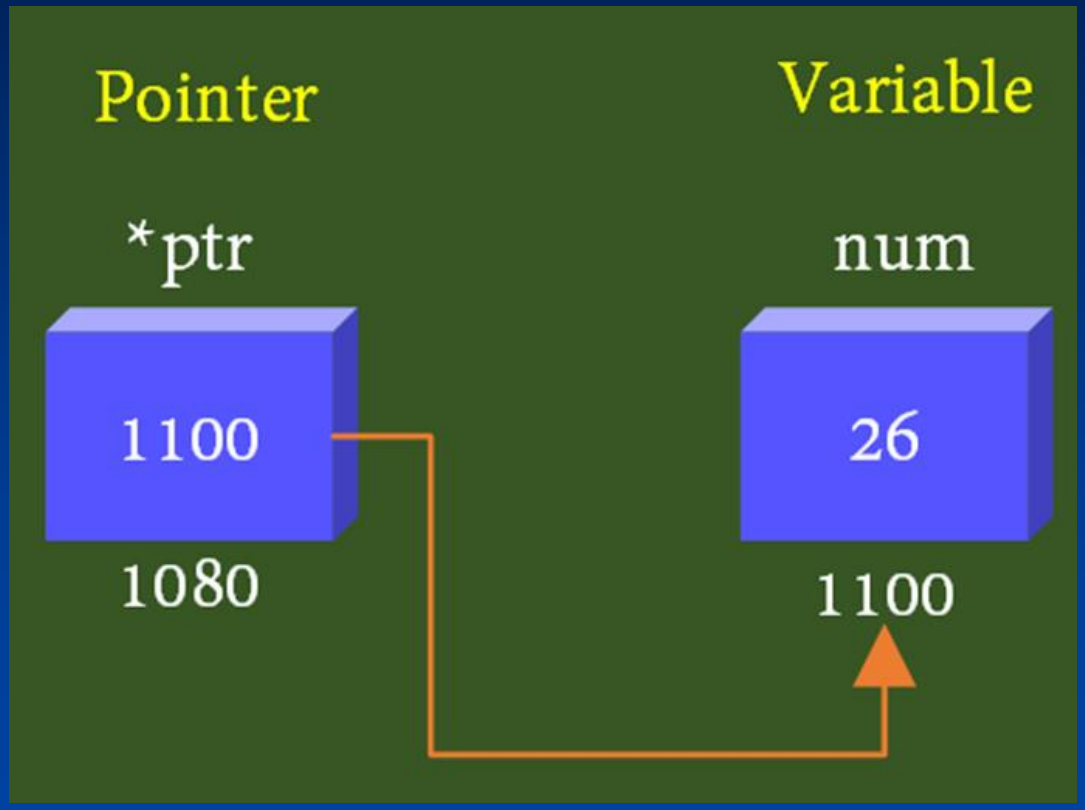



```
*(&i)=*(&i)+1;
```

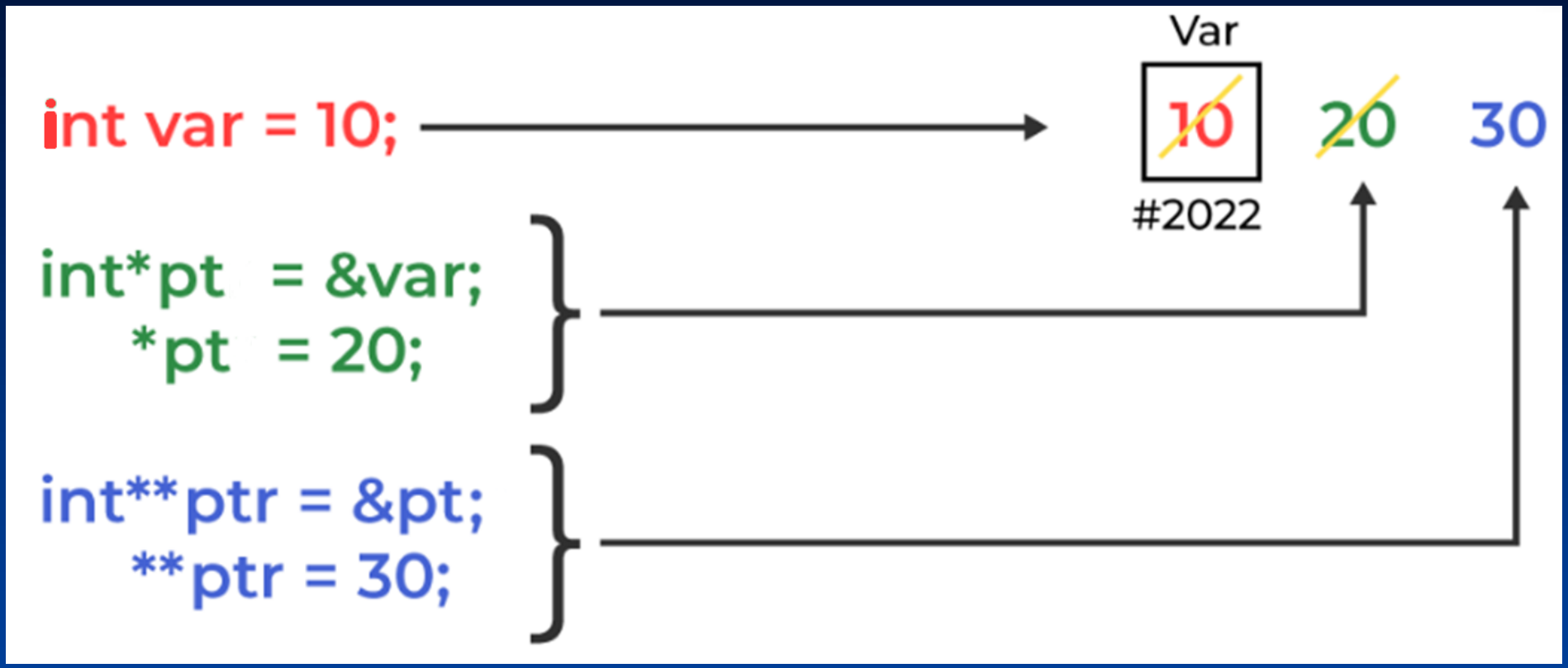
is a clumsy notation for `i++;`

Pointers : example

```
int num = 26;
int *ptr = &num;
```



Pointers and aliasing



`*pt`, `**ptr` are the same as `var`.
We call them *aliases* of `var`.

Pointers and aliasing

```
int main (int argc, char *argv[]) {
  int *p, q; // p is a pointer, q is a normal int

  q = 1;
  printf ("q=%d\n", q);

  *(&q) = *(&q) + 1;
  printf ("q=%d\n", q);

  p = &q;
  *p = *p + 1;
  printf ("q=%d\n", q);

  return 0;
}
```

Output:

q=1

q=2

q=3

Pointers as function parameters

A function can have pointer parameters.

You can use this to 'simulate' a parameter passing mechanism that is known as *call-by-reference*.

```
void nextInt(int *n) {
  (*n)++;
  printf("%d\n", *n);
}

int main(int argc, char *argv[]) {
  int n = 1;
  printf("%d\n", n);
  nextInt(&n);
  printf("%d\n", n);
  return 0;
}
```

```
Output:

1

2

2
```

An Exercise

- What is the output produced by this code?

```
void func (int a, int *b) {
  a = 7 ;
  *b = a ;
  b = &a ;
  *b = 4 ;
  printf("%d, %d\n", a, *b) ;
}

int main(int argc, char *argv[]) {
  int m = 3, n = 5;
  func(m, &n) ;
  printf("%d, %d\n", m, n) ;
  return 0;
 }
```

Output:

4, 4

3, 7

Swapping ints

We now know how to make a function that swaps two `int`s. We need to use call-by-reference!

```
void swapInts(int *pa, int *pb) {
  int h = *pa;
  *pa = *pb;
  *pb = h;
}
```

Of course, you need to call the function as follows:

```
swapInts(&a, &b);
```

Swapping two int pointers

We can repeat this technique to implement the swapping of two `int`-pointers. We use again call-by-reference!

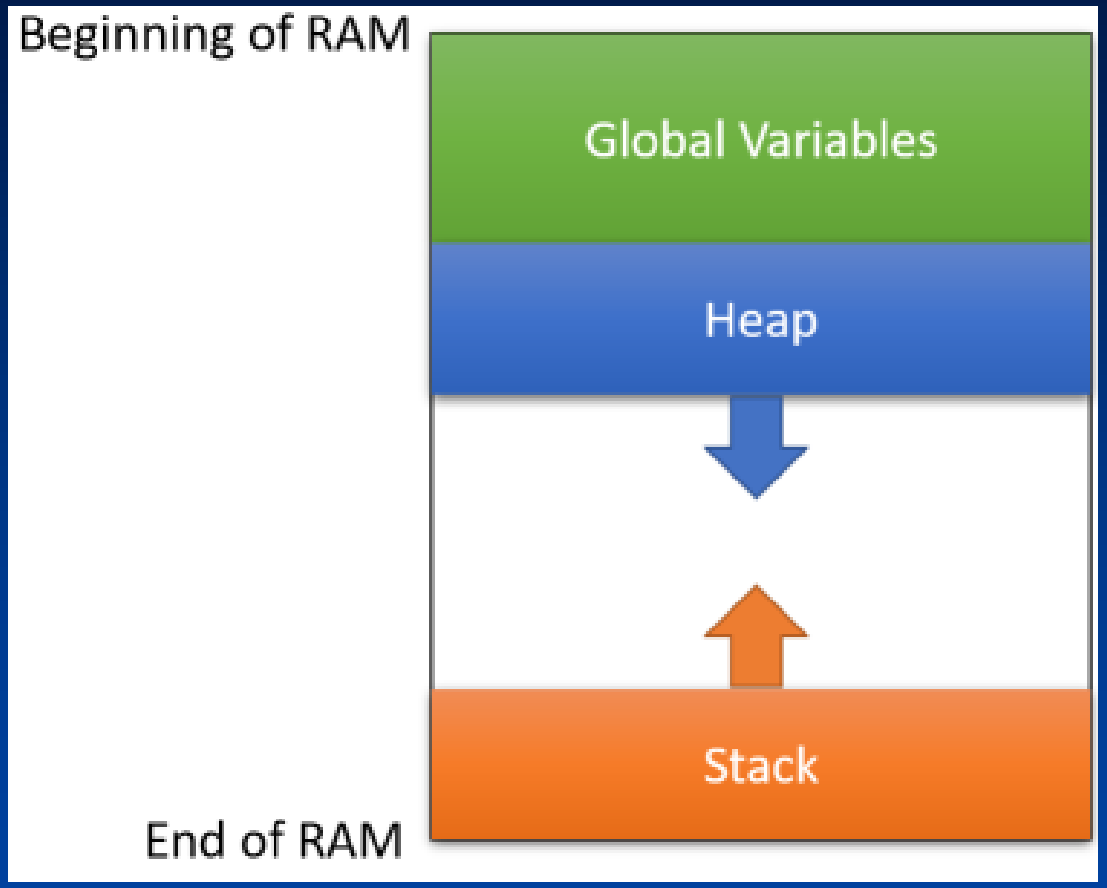
```
void swapIntPointers(int **ppa, int **ppb) {
  int *h = *ppa;
  *ppa = *ppb;
  *ppb = h;
}
```

Of course, we need to call the function as follows:

```
swapIntPointers(&pa, &pb);
```

where `pa` and `pb` are of the type `int*`

Memory layout



Local variables and arguments to functions are *pushed* on the stack. So, by calling a function, the stack 'grows'.

When a function returns these variables and arguments are removed (*popped*). So, on return from a function the stack 'shrinks'.

Passing Arrays to Functions

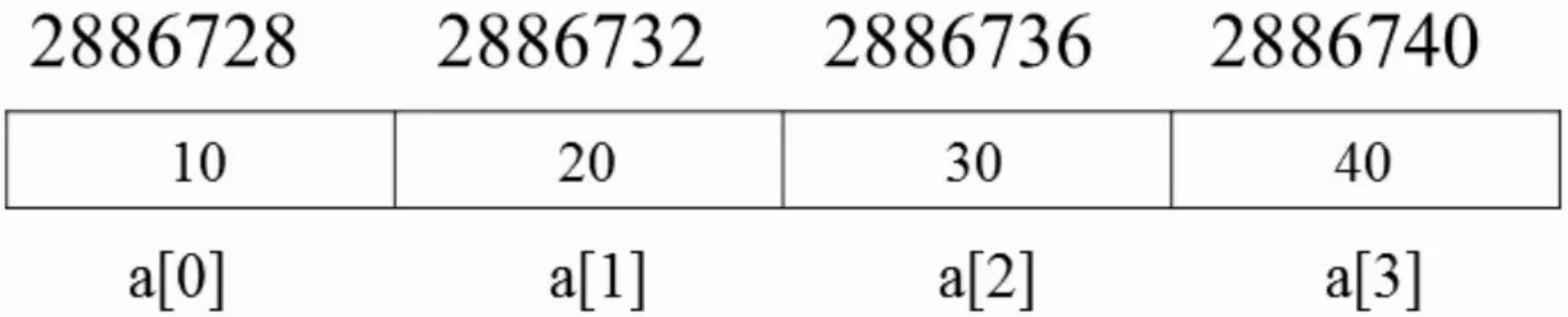
When we call a function that has an array argument, only a pointer (reference) to the first element of the array is pushed on the stack.

This avoids excessive copying of data on the stack.

Passing Arrays to Functions

```
void someFunction(int arr[]);
```

int a [ ] = {10,20,30,40};



When we call `someFunction(a)` then only `&a[0]` (which is address 2886728) is pushed on the stack.

Passing Arrays to Functions

Arrays are always passed by **reference**

- Modifications to the array are reflected to the caller!

The array name is the *address* of the first element

- If `a` is an array, than the name `a` refers actually to `&a[0]`

Since only the address of an array is passed, we often also need to pass the *size* of an array as an argument to the function.

Passing Arrays to Functions

So, typically we will write functions like:

```
void someFunction(int length, int arr[]);
```

Which is equivalent with

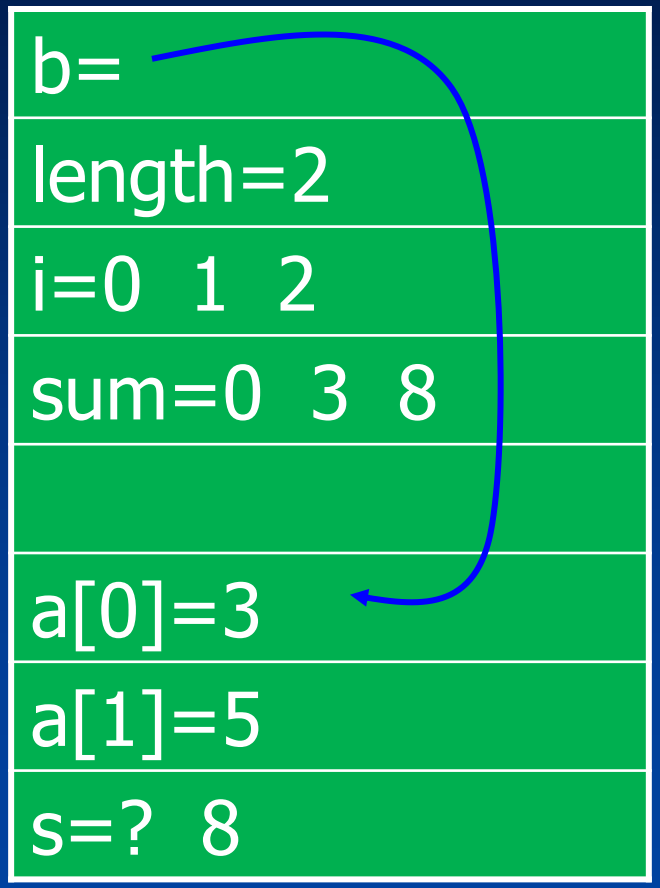
```
void someFunction(int length, int *arr);
```

Modifications of the array arr are reflected to the caller!

Example: summing an array

```
int arraySum(int length, int b[]) {
    int sum=0;
    for(int i=0; i < length; i++) {
      sum += b[i];
    }
    return sum;
}

int main(int argc, char *argv[]) {
   int a[2]={3, 5};
   int s = arraySum(2,a);
   printf("sum is %d\n", s);
   return 0;
}
```



```
void printArray(int length, int a[]) {
   printf("%d", a[0]);
   for (int i=1; i < length; i++) {
     printf(", %d", a[i]);
   }
   printf("\n");
 }

 void increment1(int a) {
   a++;
 }

void increment2(int a[]) {
   a[0]++;
 }

void increment3(int length, int a[]) {
   for (int i=0; i < length; i++) {
     a[i]++;
   }
 }
```

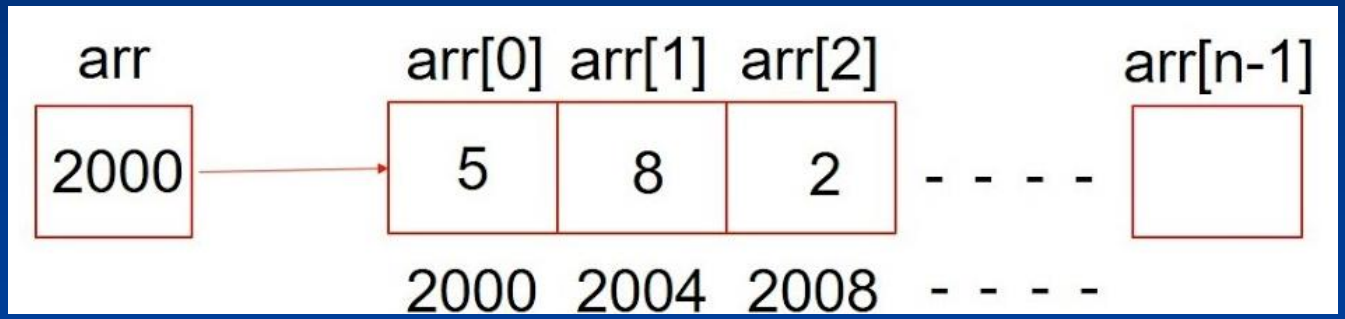
```
int a[] = {0,1,2,3,4,5,6};
int x=1;

printf("%d\n", x);
increment1(x);
printf("%d\n", x);
printArray(7, a);
increment1(a[0]);
printArray(7, a);
increment2(a);
printArray(7, a);
increment3(7, a);
printArray(7, a);
```

```
1
1
0 1 2 3 4 5 6
0 1 2 3 4 5 6
1 1 2 3 4 5 6
2 2 3 4 5 6 7
```

Passing Arrays to Functions

```
int findValue(int value, int length, int a[]) {
    for(int i=0; i < length; i++) {
      if (a[i] == value) {
        return i;
      }
    }
    return -1;
}
```




```
int arr[1000];
// assume the array is filled with data
int idx = findValue(2, 1000, arr); // passes address 2000
// idx == 2
```

Example: smallest element

```
int printSmallestElements(int length, int *arr) {
  int minimum = arr[0];
  for (int index=1; index < length; index++) {
    if (arr[index] < minimum) {
      minimum = list[index];
    }
  }

  printf("Minimum: %d\n", minimum);

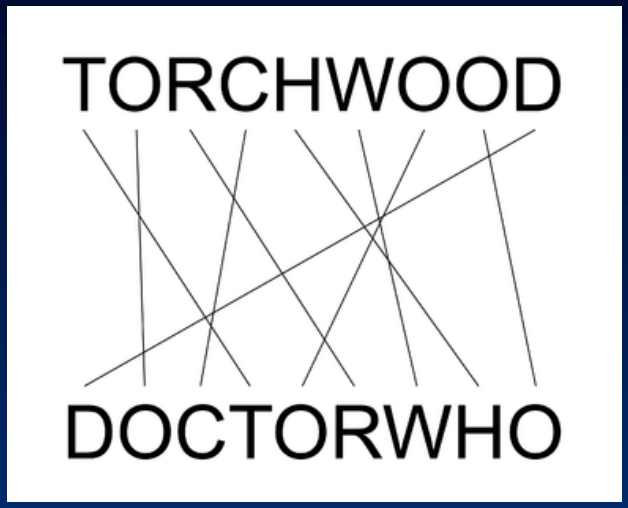
  printf("Location(s):");
  for (int index=0; index < length; index++) {
    if (arr[index] == minimum){
      printf(" %d", index);
    }
  }
  printf("\n");
  return minimum;
}
```

min/max of an array

```
void minMax(int length, int arr[], int minmax[2]) {
  minmax[0] = arr[0];
  minmax[1] = arr[0];
  for (int i=1; i < length; i++) {
    minmax[0] = (arr[i] < minmax[0] ? arr[i] : minmax[0]);
    minmax[1] = (arr[i] > minmax[1] ? arr[i] : minmax[1]);
  }
}
```

```
int a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
int extrema[2];
minMax(8, a, extrema);
printf ("Minimum: %d\n", extrema[0]);
printf ("Maximum: %d\n", extrema[1]);
```

Anagram/permutation?



Given two character arrays of length 100:

```
char a[100], b[100];
```

We assume that both arrays already contain uppercase letters ('A'..'Z').

Task: Write a function

`isAnagram(int length, char a[], char b[])`
that returns 1 if `a[]` is a *permutation* of `b[]`.

Permutations

ABCD	ABDC	ACBD
ACDB	ADBC	ADCB
BACD	BADC	BCAD
BCDA	BDAC	BDCA
CABD	CADB	CBAD
CBDA	CDAB	CDBA
DABC	DACB	DBAC
DBCA	DCAB	DCBA

4!=4 x 3 x 2 x 1 = 24 permutations

Anagram/Permutation?

```
int isAnagram(int length, char a[], char b[]) {
  int i, histogramA[26], histogramB[26];
  for (i=0; i<26; i++) {/* initialize histograms */
    histogramA[i] = 0;
    histogramB[i] = 0;
  }
  for (i=0; i<length; i++) { /* count occurrences in a[] and b[] */
    histogramA[a[i]-'A']++; /* Note the indexing! e.g 'C'-'A'== 2 */
    histogramB[b[i]-'A']++;
  }
  /* check if histogramA[i] == histogramB[i] for all i */
  for (i=0; i<26; i++) {
    if (histogramA[i]!= histogramB[i]) {
      return 0;
    }
  }
  return 1;
}
```

Anagram/Permutation?

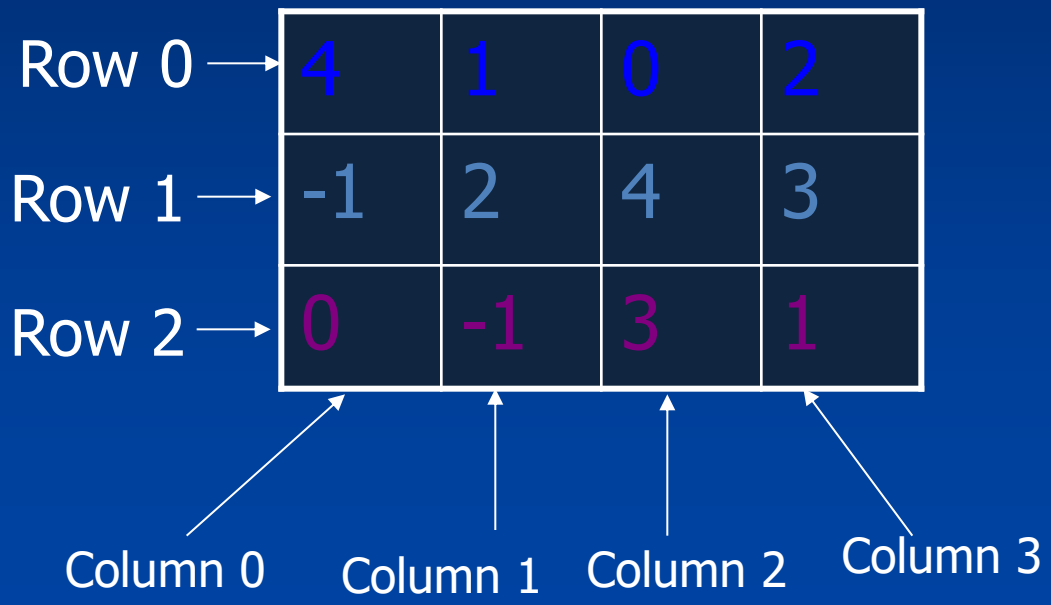
```
int isAnagram(int length, char a[], char b[]) {
  int i, diff[26];
  /* initialize histogram h[] */
  for (i=0; i<26; i++) {
    diff[i] = 0;
  }
  for (i=0; i<length; i++) {
    diff[a[i]-'A']++;
    diff[b[i]-'A']--;
  }
  for (i=0; i<26; i++) {  /* check if diff[i] == 0 for all i */
    if (diff[i]!= 0) {
      return 0;
    }
  }
  return 1;
}
```

Multi-dimensional arrays

The element type of an array can be any type.

So, the element type can itself be an array type.
This results in multi-dimensional arrays (for example, a matrix).

```
int matrix[3][4];
```



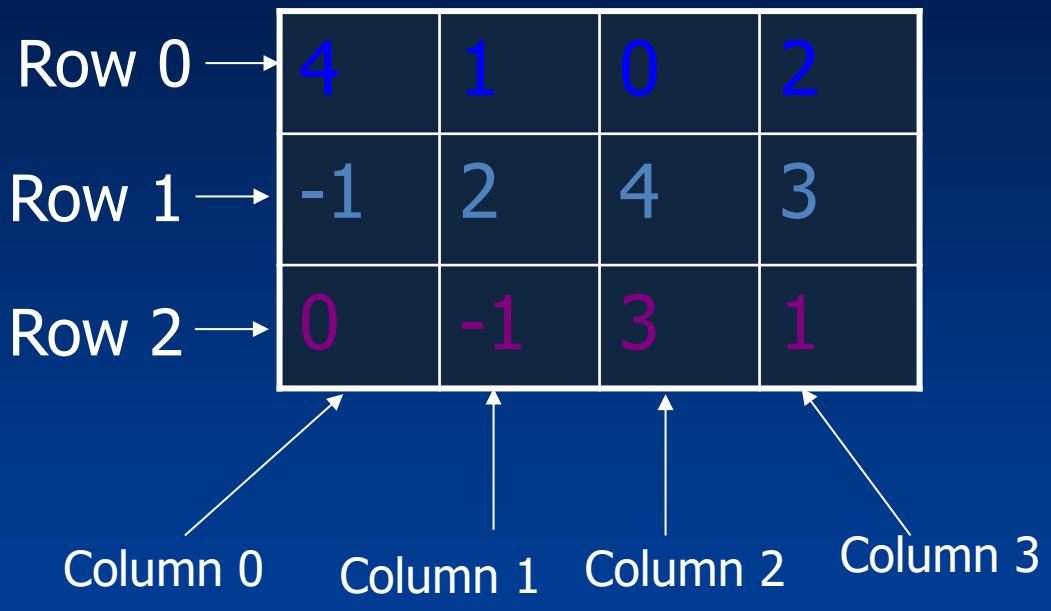
in memory



Initialization of a matrix

```
int x[3][4] = {
   {4, 1, 0, 2},
   {-1, 2, 4, 3},
   {0, -1, 3, 1}
};

int x[][4] = {
   {4, 1, 0, 2},
   {-1, 2, 4, 3},
   {0, -1, 3, 1}
};
```



Multi-dimensional arrays

```
int x[3][4];
```

A matrix is an array of arrays.

So, `x[1]` is an array of 4 elements (type `int[]`).

This is the reason why we write `x[i][j]`, and NOT `x[i,j]`.

Exercise

Write a nested loop to initialize a 2D array as follows

0	1	2
1	2	3
2	3	4
3	4	5

```
int x[4][3];
for(int i=0; i<4; i++){
   for(int j=0; j<3; j++) {
      x[i][j] = i + j;
   }
}
```

Summing rows of a matrix

```
int arraySum(int length, int a[]) { /* returns sum of 1D array */
   int sum=0;
   for(int i=0; i < length; i++) {
     sum += a[i];
   }
   return sum;
}

int main(int argc, char *argv[]) {
  int x[3][4] = {
    {4, 1, 0, 2},
    {-1, 2, 4, 3},
    {0, -1, 3, 1}
  };
  for (int i=0; i<3; i++) {
    printf("sum in row %d is %d\n", i, arraySum(4, x[i]));
  }
  return 0;
}
```

Swapping rows in a matrix

```
void swapArrays(int length, int a[], int b[]) {
  for (int i=0; i < length; i++) {
    int h = a[i];
    a[i] = b[i];
    b[i] = h;
  }
}

void swapRows(int a, int b, int matrix[3][4]) {
  swapArrays(4, matrix[a], matrix[b]);
}

int a[3][4] = {
   { 1,  0, 12, -1},
   { 7, -3,  2,  5},
   {-5, -2,  2,  9}
};
```

We can swap *the contents* of rows 0 and 2 using: `swapRows(0, 2, a);`

Note that we can not swap columns this way! This is the result of the storage scheme that is used in C: 2D-arrays are stored row-wise in memory.

Summing rows of a matrix

```
int arraySum(int length, int a[]) { /* returns sum of 1D array */
   int sum=0;
   for(int i=0; i < length; i++) {
     sum += a[i];
   }
   return sum;
}

int main(int argc, char *argv[]) {
  int x[3][4] = {
    {4, 1, 0, 2},
    {-1, 2, 4, 3},
    {0, -1, 3, 1}
  };
  for (int i=0; i<3; i++) {
    printf("sum in row %d is %d\n", i, arraySum(4, x[i]));
  }
  return 0;
}
```

Sudoku checker

Check whether a matrix of 9x9 numbers is a completed sudoku.

```
int sudoku[9][9] = {
  {7, 5, 1, 8, 4, 3, 9, 2, 6},
  {8, 9, 3, 6, 2, 5, 1, 7, 4},
  {6, 4, 2, 1, 7, 9, 5, 8, 3},
  {4, 2, 5, 3, 1, 6, 7, 9, 8},
  {1, 7, 6, 9, 8, 2, 3, 4, 5},
  {9, 3, 8, 7, 5, 4, 6, 1, 2},
  {3, 6, 4, 2, 9, 7, 8, 5, 1},
  {2, 8, 9, 5, 3, 1, 4, 6, 7},
  {5, 1, 7, 4, 6, 8, 2, 3, 9}
};
```

Sudoku checker

```
int isCorrectSudoku(int sudoku[9][9]) {
  for (int r=0; r < 9; r++) {
    for (int c = 0; c < 9; c++) {
      int digit = sudoku[r][c];
      if ((digit < 1) || (digit > 9) ||
          (cntRow(r, digit, sudoku) != 1) ||
          (cntColumn(c, digit, sudoku) != 1) ||
          (cntBlock(r, c, digit, sudoku) != 1)) {
        return 0;  /* FALSE */
      }
    }
  }
  return 1;  /* TRUE */
}
```

Sudoku checker

```
int cntRow(int row, int digit, int sudoku[9][9]) {
  int cnt = 0;
  for (int column = 0; column < 9; column++) {
    cnt += (sudoku[row][column] == digit);
  }
  return cnt;
}

int cntColumn(int column, int digit, int sudoku[9][9]) {
  int cnt = 0;
  for (int row = 0; row < 9; row++) {
    cnt += (sudoku[row][column] == digit);
  }
  return cnt;
}
```

Sudoku checker

```
int cntBlock(int row, int column, int digit, int sudoku[9][9]) {
  int cnt = 0;
  int r0, c0; /* upper left corner of block */
  r0 = row - row%3;          /* or r0 = 3*(row/3);    */
  c0 = column - column%3;    /* or c0 = 3*(column/3); */
  for (row = r0; row < r0 + 3; row++) {
    for (column = c0; column < c0 + 3; column++) {
      cnt += (sudoku[row][column] == digit);
    }
  }
  return cnt;
}
```



End week 3